

CSE 554: Systems for Machine Learning– Assignment 3

In this assignment, you will profile the performance of operations used in transformers and the transformer as a whole. The assignment consists of three parts: GEMM profiling, Attention Profiling, and Transformer Profiling. Your submission will be in the form of a .zip file upload on Canvas, and must include a **short report with answers to the following questions (.pdf)** and the **code**.

- **Note:** When compiling libraries, please use make with “-j 32” rather than “-j” only. This would limit CPU usage and avoid crashing making the machine unresponsive.
- For both matrix multiplication and attention, we use FP16 data type.

Section 1: General Matrix Multiplication

Q1. 2 + 5 + 3 points

- What is the operational intensity of a GEMM in terms of M, N, K dimensions?
- Calculate the compute intensity for M in 128 - 2048 with step 128, (N, K) = (512, 512), (4096, 4096), (14336, 4096), (4096, 1024), (1024, 4096) and show the result in a table. You can use <https://www.tablesgenerator.com> to generate tables.

M	(512, 512)	(4096, 4096)	(14336, 4096)	(4096, 1024)	(1024, 4096)
128					
...					
2048					

- What is CUTLASS? What is CUBLAS? What is the key difference between them?
 - <https://docs.nvidia.com/cuda/cublas/>
 - <https://github.com/NVIDIA/cutlass>

Q2. 5 + 5 + 3 + 2 points

- Profile the CUBLAS cublasGEMMEx performance for the above shapes and plot the performance. Plot one figure for one shape with the M dimension on the x-axis and TFLOPs (total compute/time) on the y-axis.
 - Please use the profiling methods in [Section1/prof_example.cu](#). We do warmups and L2 cache flush to increase the profiling accuracy.
 - The example plotting script is given at [Section1/plot_gemm.py](#).
- Profile the CUTLASS GEMM performance using the CUTLASS profiler for the above shapes. The cutlass profiler will iterate through all kernel implementations with different

tile sizes and configurations. Use again the `--profiling-iterations` of 100. Profile different `--split_k_slices=1,2,4,8` with `--split_k_mode=serial`. Split K will control the number of splits in K dimension. For example, for GEMM shape (128, 1024, 4096), when split K is 4, the kernel will first treat it as 4 GEMM problems of size (128, 1024, 1024) and perform a reduction to add the partial results. Take the best performance across all implementations for each GEMM shape. Plot the CUTLASS performance in the same figure as CUBLAS.

- <https://github.com/NVIDIA/cutlass/blob/main/media/docs/cpp/quickstart.md>
 - `$ cmake .. -DCMAKE_CUDA_ARCHITECTURES=75`
`-DCUTLASS_ENABLE_TESTS=OFF -DCUTLASS_ENABLE_EXAMPLES=OFF`
`-DCUTLASS_NVCC_ARCHS=75`
 - `$ cmake --build . -j 32`
 - Investigate the kernel that achieves the best performance in CUTLASS. What is the relationship between problem size, tile size, and split K?
 - https://github.com/NVIDIA/cutlass/blob/main/media/docs/cpp/gemm_api_3x.md
 - What is the difference between the CUTLASS performance and the CUBLAS performance for different shapes? Describe your findings in a few sentences.
-

Section 2: Attention

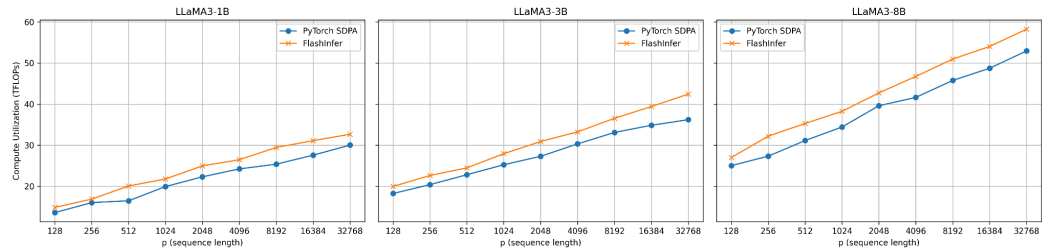
Q1. 5 + 5 points

- Given model `num_kv_heads`, `num_qo_heads`, `head_dim`, what is the operational intensity of
 - Prefill attention, when the prompt length is `p`
 - Decode attention, when the context length is `c`
- Compute the operational intensity for llama3-1B, llama3-3B, llama3-8B
 - For prefill attention, $p = 2^7, 2^8, \dots, 2^{15}$
 - For decode attention, $c = 2^7, 2^8, \dots, 2^{15}$

Q2. 10 + 10 points

- Install Flashinfer using `uv pip install flashinfer-python==0.5.3`
- Evaluate the prefill attention performance of one layer using [torch.nn.functional.scaled_dot_product_attention](#) and [FlashInfer](#) for
 - Batch size = 1, $p = 2^7, 2^8, \dots, 2^{15}$. Plot the compute utilization TFlops of each model. Use $\log_2(p)$ as the x-axis. Generate one **subplot per model** inside a shared figure. In each subplot, draw line plots for both torch SDPA and FlashInfer using different line styles or colors. Label axes clearly. (see [Section2/plot_attention_p.py](#))

Prefill Attention Compute Utilization (Fake Data)



-
- Batch size = $2^0, 2^1, \dots, 2^6$, $p = 1024$ for all three models. Plot compute utilization using log batch size as the x-axis.
- Evaluate the decode attention performance of one layer using torch SDPA and FlashInfer
 - Batch size = 1, $c = 2^7, 2^8, \dots, 2^{15}$. Plot memory bandwidth utilization (GB/s) similar to previous questions.
 - Batch size = $2^0, 2^1, \dots, 2^6$, $c = 1024$ for all three models. Plot memory bandwidth utilization similarly.
 - Batch size = 128, $c = 1024$, page size = 1, 2, 4, 8, 16 for all three models using FlashInfer. Plot the memory bandwidth utilization of each model.

Section 3: End-to-end performance

Q1. 15 points

- Starting from the [Section3/flashinfer_pipeline.py](#), construct a serving engine using FlashInfer attention and rope kernels with paged attention.

Q2. 10 + 10 + 10 points

- With batch size 32, prefill length 256, decode length $2^5 - 2^{10}$, profile the prefill time and total decode time. Plot end-to-end time curve with the $\log(\text{decode length})$ as the x-axis. Which phase is the key bottleneck? In the last decode cycle, which operation takes the longest time for different decode lengths?
- With batch size 1, prefill length $2^8 - 2^{14}$, profile the prefill time. Plot end-to-end time curve with $\log(\text{prefill length})$ as x-axis. Examine the prefill time breakdown. What operations are the dominating factor for various prefill lengths?
- With batch size $2^0 - 2^8$, prefill length 128, decode length 128, plot the end-to-end time curve with $\log(\text{batch size})$ as x-axis. Plot the total throughput $((\text{prefill} + \text{decode}) / \text{time})$ curve with $\log(\text{batch size})$ as x-axis. When does the performance saturate?